

UNIT-1 ANSWERS:

PART-A

Define an Operating System.

An **Operating System (OS)** is system software that acts as an interface between the user and the computer hardware. It manages hardware resources and provides services for application programs.

2. List the objectives of an Operating System.

The main objectives of an Operating System are:

- To make the computer easy to use.
- To manage hardware resources efficiently.
- To provide a platform for running application programs.
- To improve system performance.
- To ensure security and protection of data.
- To support multitasking and resource sharing.

3. Tabulate the differences between Kernel and Shell.

Kernel	Shell
Core part of the operating system.	User interface of the operating system.
Directly interacts with hardware.	Interacts with the user and communicates with the kernel.
Manages memory, CPU, and devices.	Accepts user commands and executes them.
Loads during system startup.	Starts after the kernel is loaded.
Runs in privileged mode.	Runs in user mode.

4. Define a Bootstrap Program.

A **Bootstrap Program** is a small program stored in the computer's ROM or firmware that initializes the system hardware and loads the operating system into memory during startup.

5. Recall the Multiprogramming Concept.

Multiprogramming is an operating system technique in which multiple programs are kept in the main memory at the same time, allowing the CPU to switch between them and improving CPU utilization.

6. Define Real-Time Operating System (RTOS).

A **Real-Time Operating System (RTOS)** is an operating system designed to process data and respond to events within a fixed time limit. It is commonly used in applications where timely and predictable responses are essential, such as embedded systems, robotics, and industrial control systems.

7. Define Spooling

Spooling (Simultaneous Peripheral Operations On-Line) is a technique in which data is temporarily stored in a buffer (usually on disk) so that slow input/output devices, such as printers, can process it one job at a time while the CPU continues executing other tasks.

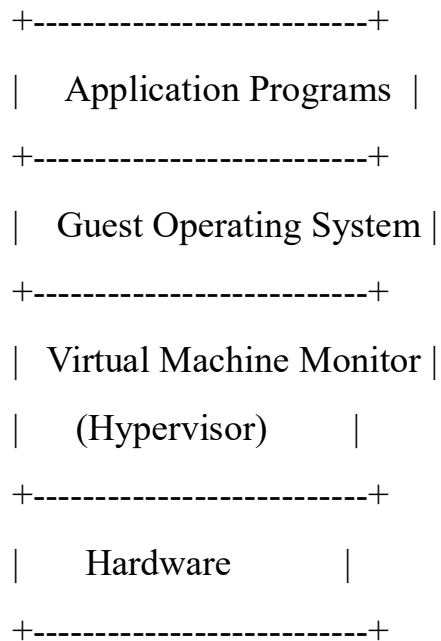
8. Tabulate the differences between Hard and Soft Real-Time Systems

Hard Real-Time System	Soft Real-Time System
Missing a deadline is unacceptable.	Missing a deadline is acceptable occasionally.
Guarantees response within a fixed time.	Tries to meet deadlines but no strict guarantee.
Used in aircraft control, medical devices.	Used in multimedia, online gaming.
High reliability and predictability.	Performance is more important than strict timing.

9. List any two advantages of Microkernel Architecture

- **Better security and reliability** because only essential services run in the kernel.
 - **Easy to maintain and extend**, since services can be added or modified without changing the kernel.
-

10. Sketch the Virtual Machine Architecture



11. List the categories of System Calls

- Process Control
 - File Management
 - Device Management
 - Information Maintenance
 - Communication
 - Protection and Security
-

12. Define Command Interpreter

A **Command Interpreter (Shell)** is a program that accepts user commands, interprets them, and requests the operating system to execute them.

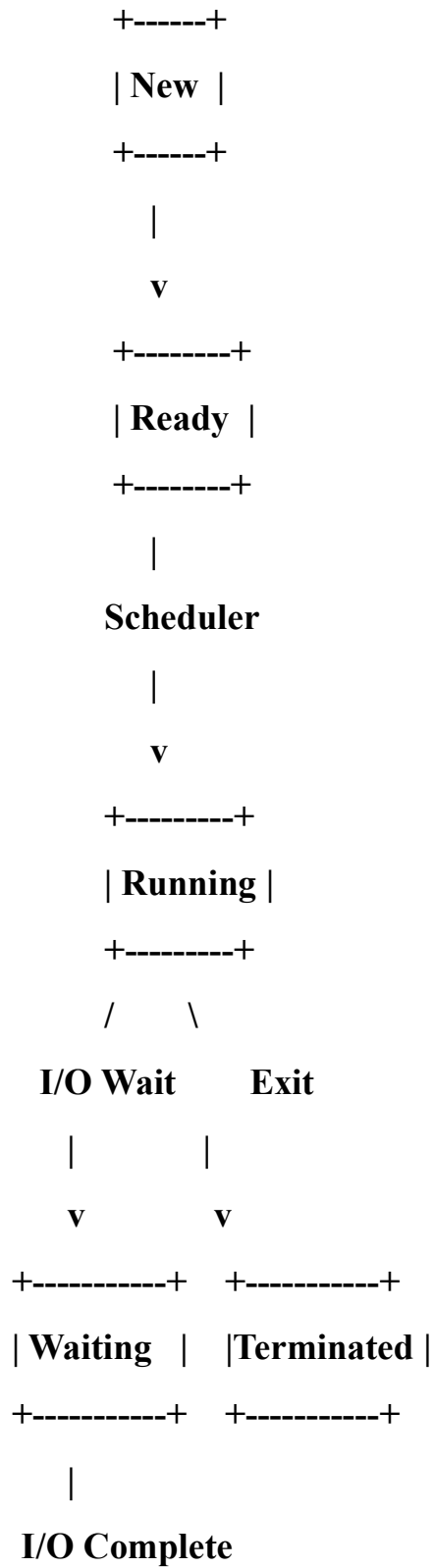
13. List some of the Operating System Services

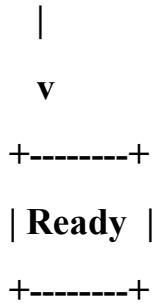
- Program execution
 - I/O operations
 - File system management
 - Memory management
 - Process management
 - Device management
 - Error detection
 - Security and protection
 - Communication between processes
-

14. Tabulate the differences between Process and Thread

Process	Thread
Independent program in execution.	Smallest unit of execution within a process.
Has its own memory space.	Shares memory with other threads of the same process.
Creation is slower.	Creation is faster.
Context switching is costly.	Context switching is faster and less costly.
Communication is slower.	Communication is easier through shared memory.

15. Draw the Five-State Process Model





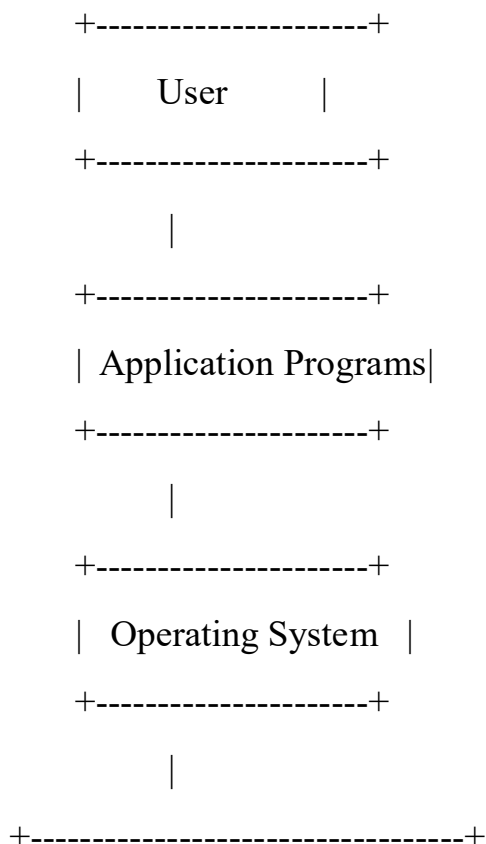
PART-B

1. Explain the Objectives and Functions of an Operating System with a Neat Diagram

Definition

An **Operating System (OS)** is system software that acts as an interface between the user and computer hardware. It manages hardware resources and provides services to application programs.

Neat Diagram



| CPU | Memory | I/O Devices | Disk |

+-----+

Objectives of an Operating System

1. **Convenience** – Makes the computer easy to use.
2. **Efficiency** – Utilizes hardware resources effectively.
3. **Resource Management** – Allocates CPU, memory, and I/O devices.
4. **Performance** – Improves system speed and throughput.
5. **Security** – Protects programs and user data.
6. **Reliability** – Ensures stable and error-free operation.

Functions of an Operating System

- Process Management
- Memory Management
- File System Management
- Device Management
- Secondary Storage Management
- Security and Protection
- Error Detection
- User Interface (Shell/GUI)
- Networking and Communication

2. Describe the Evolution of Operating Systems from Serial Processing to Distributed Systems

(a) Serial Processing (1940s–1950s)

- No operating system.
- One program executed at a time.
- Programs loaded manually.

- CPU remained idle during setup.

Disadvantages

- Slow execution.
 - Poor CPU utilization.
-

(b) Simple Batch Systems

- Similar jobs grouped into batches.
- Operator loads jobs automatically.
- Reduced setup time.

Advantages

- Better CPU utilization.
 - Automatic job sequencing.
-

(c) Multiprogramming Systems

- Multiple programs kept in memory simultaneously.
- CPU switches to another job during I/O waiting.

Advantages

- Higher CPU utilization.
 - Increased throughput.
-

(d) Time-Sharing Systems

- Many users share the CPU simultaneously.
- Each user gets a small time slice.

Advantages

- Interactive computing.

- Quick response time.

(e) Real-Time Systems

- Produces output within a specified deadline.
- Used in industrial and medical applications.

(f) Distributed Systems

- Multiple computers connected through a network.
- Resources shared among systems.

Advantages

- Resource sharing.
- High reliability.
- Scalability.
- Faster processing.

3. Discuss the Multiprogramming Operating System with a Suitable Diagram

Definition

A **Multiprogramming Operating System** keeps several programs in memory at the same time. When one program waits for I/O, the CPU executes another program, increasing CPU utilization.

Diagram

Main Memory

+-----+

| Program A |

+-----+

| Program B |

+-----+
| Program C |
+-----+
| Operating Sys. |
+-----+

CPU

|

A → Waiting for I/O

↓

CPU executes B

↓

CPU executes C

Working

- Multiple jobs are loaded into memory.
- CPU executes one job.
- If the running job requests I/O, CPU switches to another ready job.
- This process continues until all jobs finish.

Advantages

- Better CPU utilization.
- Increased throughput.
- Reduced idle time.
- Efficient use of memory.

Disadvantages

- Complex memory management.
 - Difficult scheduling.
 - Requires more memory.
-

4. Explain Hard Real-Time and Soft Real-Time Operating Systems

Hard Real-Time Operating System

A **Hard Real-Time OS** guarantees that every task is completed before its deadline. Missing a deadline may lead to system failure.

Characteristics

- Strict timing constraints.
- Highly predictable.
- Used in safety-critical systems.

Applications

- Aircraft control systems
 - Medical equipment
 - Nuclear power plants
 - Industrial robots
-

Soft Real-Time Operating System

A **Soft Real-Time OS** aims to meet deadlines, but occasional delays are acceptable.

Characteristics

- Flexible timing constraints.
- Better overall performance.
- Missing deadlines reduces quality but does not cause failure.

Applications

- Multimedia systems
- Video conferencing
- Online gaming
- Web servers

Comparison

Hard Real-Time

Deadline must never be missed.

Predictable response.

Safety-critical applications.

System failure if deadline is missed.

Soft Real-Time

Deadline may occasionally be missed.

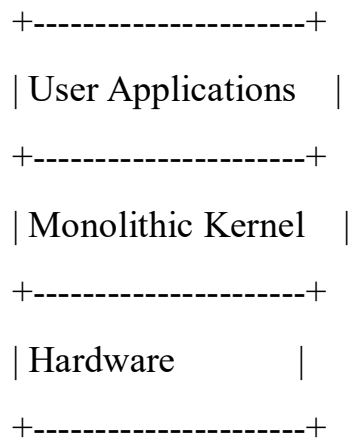
Best-effort response.

Multimedia and communication applications.

Performance degradation if deadline is missed.

5. Illustrate Different Operating System Structures with a Neat Diagram

(a) Simple (Monolithic) Structure



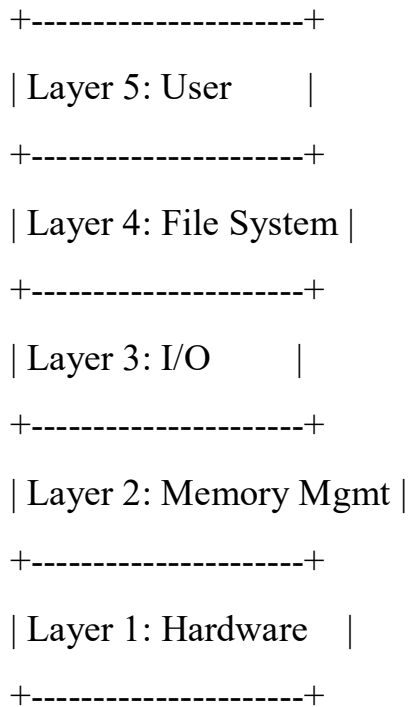
Advantages

- Fast execution.
- Simple design.

Disadvantages

- Difficult to maintain.
- Less secure

(b) Layered Structure



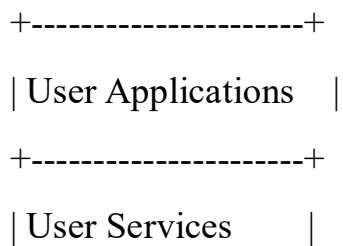
Advantages

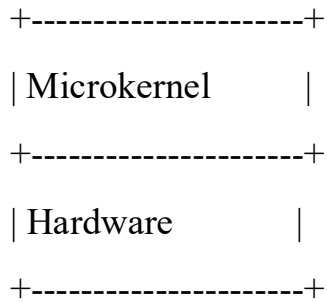
- Easy debugging.
- Better modularity.

Disadvantages

- Slower due to multiple layers.

(c) Microkernel Structure





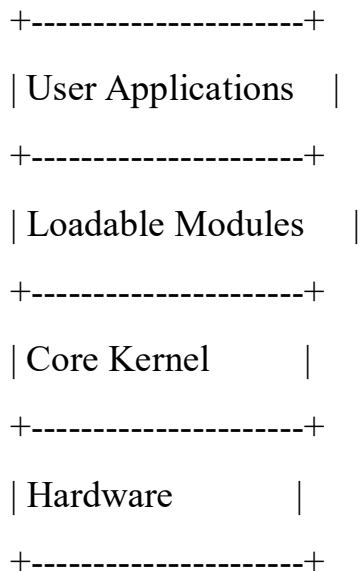
Advantages

- Secure and reliable.
- Easy to extend.

Disadvantages

- Communication overhead.

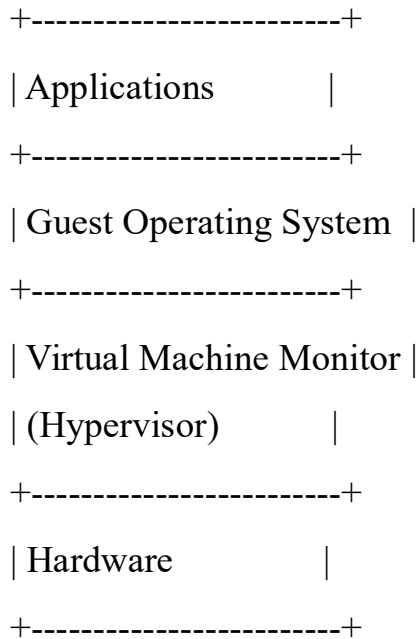
(d) Modular Structure



Advantages

- Flexible.
 - Modules can be added dynamically.
-

(e) Virtual Machine Structure



Advantages

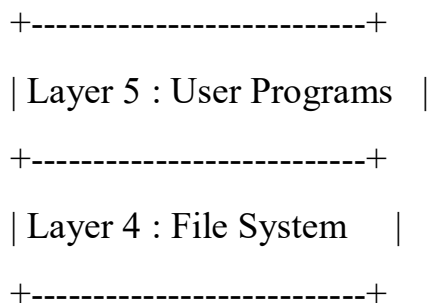
- Multiple operating systems can run simultaneously.
- Better isolation and security.

6. Discuss the Layered Operating System Architecture

Definition

The **Layered Operating System Architecture** divides the operating system into several layers. Each layer performs a specific function and communicates only with its adjacent layers. The lowest layer interacts with the hardware, while the highest layer interacts with the user.

Neat Diagram



| Layer 3 : I/O Management |

+-----+

| Layer 2 : Memory & Process|

+-----+

| Layer 1 : Hardware |

+-----+

Working

- Each layer uses the services of the layer below it.
- Each layer provides services to the layer above it.
- Communication occurs only between adjacent layers.

Advantages

- Modular design.
- Easy to debug and maintain.
- Improved security.
- Easier testing.
- Better reliability.

Disadvantages

- Slower performance due to multiple layers.
- Difficult to define proper layers.
- Increased overhead.

7. Describe the Operating System Services in Detail

The Operating System provides various services to users and application programs.

1. Program Execution

- Loads programs into memory.

- Executes and terminates programs.

2. Process Management

- Creates and deletes processes.
- Schedules CPU.
- Synchronizes processes.
- Handles deadlocks.

3. Memory Management

- Allocates and deallocates memory.
- Keeps track of memory usage.
- Supports virtual memory.

4. I/O Device Management

- Controls input/output devices.
- Provides device drivers.
- Performs buffering, caching, and spooling.

5. File System Management

- Creates and deletes files.
- Organizes directories.
- Controls file access permissions.

6. Secondary Storage Management

- Manages disk space.
- Performs disk scheduling.
- Allocates storage blocks.

7. Communication

- Enables communication between processes.
- Supports networking.

8. Error Detection

- Detects CPU, memory, and device errors.
- Maintains system stability.

9. Protection and Security

- User authentication.
- Access control.
- Data protection.

10. User Interface

- Command Line Interface (CLI)
- Graphical User Interface (GUI)

8. Explain Process Concept and Process Life Cycle with a Neat Diagram

Process Concept

A **Process** is a program in execution. It consists of program code, data, program counter, CPU registers, stack, and allocated resources.

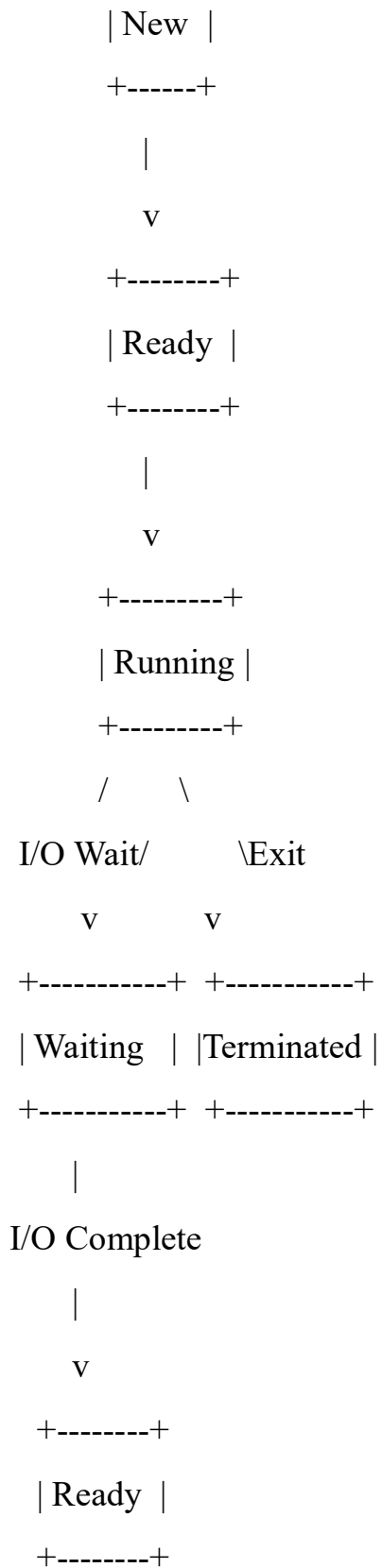
Process Components

- Program Code
- Program Counter
- CPU Registers
- Stack
- Heap
- Data Section

Process Life Cycle

Five-State Process Model

+-----+



Process States

1. New

- Process is created.

2. Ready

- Waiting for CPU allocation.

3. Running

- CPU executes the process.

4. Waiting (Blocked)

- Waiting for I/O or an event.

5. Terminated

- Process execution is completed.
-

9. Illustrate Context Switching with a Suitable Example

Definition

Context Switching is the process of saving the state (context) of the currently running process and loading the state of another process so that CPU execution can continue.

Context includes

- Program Counter
- CPU Registers
- Stack Pointer
- Process State
- Memory Information

Diagram

CPU Running Process P1

|

Timer Interrupt

|

Save Context of P1

|

Load Context of P2

|

CPU Executes P2

Example

Suppose two processes exist:

- **P1** – Word Processor
- **P2** – Music Player

Initially:

CPU → P1

When the time slice expires:

Save P1 Context

↓

Load P2 Context

↓

CPU → P2

After another time slice:

Save P2 Context

↓

Restore P1 Context

↓

CPU → P1

This enables multitasking without losing the execution state of either process.

Advantages

- Supports multitasking.
- Efficient CPU utilization.
- Better responsiveness.

Disadvantages

- Switching overhead.
- No useful work during switching.

10. Discuss Multithreading Models with Suitable Diagrams

Definition

Multithreading allows multiple threads within a single process to execute concurrently.

1. Many-to-One Model

Many user threads are mapped to one kernel thread.

User Threads

T1

T2

T3

|

v

Kernel Thread

|

Hardware

Advantages

- Easy to implement.
- Fast thread creation.

Disadvantages

- One blocking thread blocks all.
 - Cannot use multiple CPUs.
-

2. One-to-One Model

Each user thread has a corresponding kernel thread.

User Threads Kernel Threads

T1 -----> K1

T2 -----> K2

T3 -----> K3

Advantages

- True parallelism.
- Better responsiveness.

Disadvantages

- Higher overhead.
 - More kernel resources required.
-

3. Many-to-Many Model

Many user threads are mapped to many kernel threads.

User Threads

T1

T2

T3

T4

\ | /

\|/

Kernel Threads

K1

K2

|

Hardware

Advantages

- Efficient resource utilization.
- Supports parallel execution.
- Greater flexibility.

Disadvantages

- Complex implementation.
-

PART-C

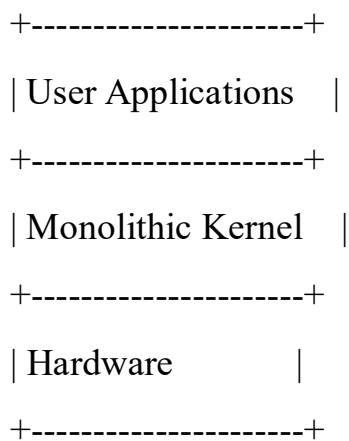
Discuss Various Operating System Structures. Which Structure is Most Suitable for Modern Operating Systems? Justify Your Answer.

Operating System Structures

(a) Monolithic Structure

In a **Monolithic Operating System**, all OS services run in a single large kernel.

Diagram



Advantages

- Fast execution.
- Simple communication between components.

Disadvantages

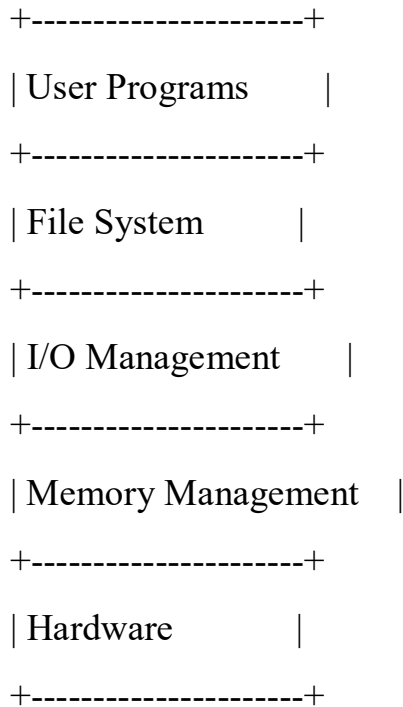
- Difficult to maintain.
- A bug in one module may crash the entire system.
- Less secure.

Examples: MS-DOS, early UNIX.

(b) Layered Structure

The OS is divided into layers, where each layer provides services to the layer above it.

Diagram



Advantages

- Modular design.
- Easy debugging and maintenance.

Disadvantages

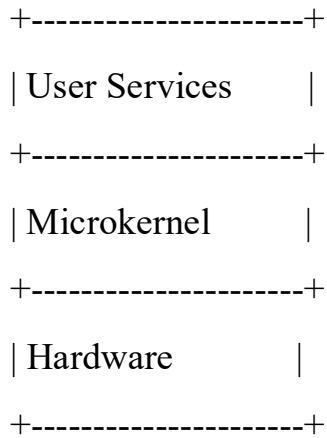
- Performance overhead due to multiple layers.

(c) Microkernel Structure

Only essential functions run in the kernel. Other services execute in user space.

Diagram





Advantages

- High security.
- Better reliability.
- Easy to extend.

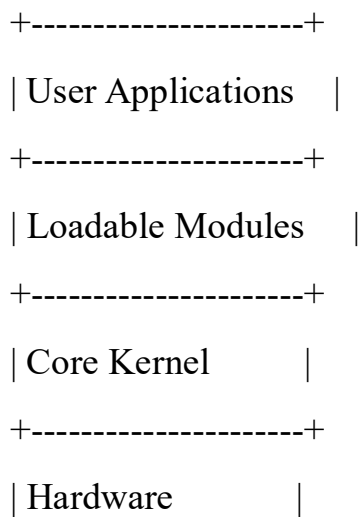
Disadvantages

- Communication overhead between user services and kernel.

(d) Modular Structure

The kernel consists of a core with dynamically loadable modules.

Diagram



+-----+

Advantages

- Flexible.
- Easy to update.
- Better performance than microkernel.

(e) Virtual Machine Structure

A hypervisor allows multiple operating systems to run on the same hardware.

Diagram

+-----+

| Guest Operating Systems |

+-----+

| Hypervisor |

+-----+

| Hardware |

+-----+

Advantages

- Isolation.
- Better resource utilization.
- Supports multiple OSs.

Which Structure is Most Suitable for Modern Operating Systems?

Answer: Modular Structure is the most suitable for modern operating systems.

Justification

- High performance because most services run in kernel space.
- Modules can be loaded or removed without restarting the system.
- Easier maintenance and upgrades.
- More reliable than monolithic systems.
- Supports modern hardware and device drivers efficiently.

Examples: Linux, Solaris.

2. Assume You Are Designing an Operating System for an ATM or Aircraft Control System. Which Type of Operating System Would You Choose? Justify Your Answer.

Answer

I would choose a **Hard Real-Time Operating System (Hard RTOS)**.

Reason

ATMs and aircraft control systems perform safety-critical operations where responses must occur within strict deadlines. Missing a deadline can result in incorrect transactions or even endanger lives.

Features of Hard RTOS

- Guaranteed response within a fixed time.
- Deterministic scheduling.
- High reliability.
- Predictable execution.
- Fast interrupt handling.

ATM Example

- Card authentication
- PIN verification
- Cash dispensing
- Account update

All these operations must complete correctly and promptly.

Aircraft Control Example

- Flight control
- Engine monitoring
- Navigation
- Collision avoidance

Even a small delay can have serious consequences.

Advantages

- Meets strict deadlines.
- Highly reliable.
- Supports mission-critical applications.
- Provides deterministic behavior.

Conclusion

A **Hard Real-Time Operating System** is the best choice because it guarantees timely and predictable execution, which is essential for ATM transactions and aircraft control.

3. A University Wants to Upgrade Its Computer Laboratory from a Batch Processing System to a Multitasking System. Compare Both Systems and Explain the Benefits of Migration.

Comparison of Batch Processing and Multitasking Systems

Batch Processing System

Jobs are executed one after another.

No user interaction during execution.

Multitasking System

Multiple programs run simultaneously.

Interactive user environment.

Batch Processing System

CPU may remain idle during I/O.

Slow response time.

Lower CPU utilization.

Suitable for payroll and billing.

Multitasking System

CPU switches to another task during I/O.

Fast response time.

Higher CPU utilization.

Suitable for laboratories and personal computers.

Benefits of Migrating to a Multitasking System

1. Better CPU Utilization

The CPU executes another program while one program waits for I/O, reducing idle time.

2. Faster Response

Students can switch between applications without waiting for one task to finish.

3. Improved Productivity

Multiple applications such as programming tools, web browsers, and office software can run simultaneously.

4. Better Resource Sharing

Memory, CPU, printers, and storage are shared efficiently among users and applications.

5. Interactive Computing

Students receive immediate feedback while compiling programs, browsing the internet, or editing documents.

6. Increased Throughput

More work is completed in the same amount of time.

Example

Batch Processing

- Student waits until a compilation job completes before starting another task.

Multitasking

- Student can compile a program, browse study material, listen to lectures, and prepare documentation simultaneously.
-

Conclusion

Migrating from a **Batch Processing System** to a **Multitasking System** significantly improves the efficiency and usability of a university computer laboratory. It enhances CPU utilization, reduces waiting time, supports interactive learning, and allows students to work on multiple tasks at the same time, making it the preferred choice for modern educational environments.

UNIT-II ANSWERS:

PART-A

1. Define CPU Scheduling

CPU Scheduling is the process of selecting one process from the ready queue and allocating the CPU to it for execution.

2. List the Scheduling Criteria Used to Evaluate CPU Scheduling Algorithms

The common scheduling criteria are:

- **CPU Utilization** – Maximize CPU usage.
- **Throughput** – Maximize the number of processes completed per unit time.
- **Turnaround Time** – Minimize the total time from submission to completion.
- **Waiting Time** – Minimize the time a process waits in the ready queue.

- **Response Time** – Minimize the time taken to respond to a user's request.
 - **Fairness** – Ensure every process gets a fair chance to execute.
-

3. State One Advantage and One Disadvantage of FCFS Scheduling

Advantage:

- Simple and easy to implement.

Disadvantage:

- Suffers from the **Convoy Effect**, where short processes wait behind long processes, increasing average waiting time.
-

4. Define Shortest Job First (SJF) Scheduling

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the **smallest CPU burst time** for execution first.

Advantage:

- Produces the minimum average waiting time.

Disadvantage:

- Difficult to predict the exact CPU burst time.
-

5. Tabulate the Differences Between Preemptive and Non-Preemptive SJF Scheduling

Preemptive SJF (SRTF)	Non-Preemptive SJF
CPU can be taken away from the running process.	CPU cannot be taken away until the process finishes.
Also called Shortest Remaining Time First (SRTF) .	Simply called Shortest Job First (SJF) .

Preemptive SJF (SRTF)

Better response time.

More context switches.

More complex to implement.

Non-Preemptive SJF

Higher waiting time for newly arrived short jobs.

Fewer context switches.

Easier to implement.

6. Define Priority Scheduling

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority, and the CPU is allocated to the process with the **highest priority** first.

Note: It may lead to **starvation** of low-priority processes.

7. Illustrate What Happens if the Time Quantum is Very Small

In **Round Robin Scheduling**, if the **time quantum is very small**:

- The CPU switches between processes very frequently.
- The number of context switches increases.
- CPU spends more time switching than executing processes.
- System performance decreases due to high overhead.

Illustration

Time Quantum = Very Small

P1 → P2 → P3 → P1 → P2 → P3 → ...

(Frequent Context Switching)

8. Define Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism that allows two or more processes to **exchange data and synchronize their activities**.

Common IPC Methods

- Shared Memory
- Message Passing
- Pipes
- Sockets
- Signals

Purpose:

- Data sharing
- Process synchronization
- Communication between cooperating processes

PART-B

1. Explain FCFS Scheduling Algorithm with a Suitable Example

Definition

First Come First Serve (FCFS) is a **non-preemptive CPU scheduling algorithm** in which the process that arrives first gets the CPU first. Processes are executed in the order of their arrival.

Characteristics

- Simple and easy to implement.
- Uses a FIFO (First In First Out) queue.
- Once a process gets the CPU, it runs until completion.

Example

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3

Process	Arrival Time	Burst Time
P3	2	8
P4	3	6

Gantt Chart

0 5 8 16 22
 |---P1---|--P2--|----P3----|---P4----|

Waiting Time

Process	Waiting Time
P1	0
P2	$5 - 1 = 4$
P3	$8 - 2 = 6$
P4	$16 - 3 = 13$

Average Waiting Time

$$= (0 + 4 + 6 + 13) / 4 = \mathbf{5.75 \text{ ms}}$$

Turnaround Time

Process	Turnaround Time
P1	5
P2	7
P3	14
P4	19

Average Turnaround Time

$$= (5 + 7 + 14 + 19) / 4 = \mathbf{11.25 \text{ ms}}$$

Advantages

- Simple implementation.

- Fair scheduling.

Disadvantages

- Convoy effect.
- High waiting time for short jobs.

2. Discuss the SJF Scheduling Algorithm with Gantt Chart and Calculate Waiting Time and Turnaround Time

Definition

Shortest Job First (SJF) selects the process with the **smallest CPU burst time** for execution first.

Example

Process	Arrival Time	Burst Time
P1	0	6
P2	0	8
P3	0	7
P4	0	3

Execution Order

P4 → P1 → P3 → P2

Gantt Chart

0 3 9 16 24
 |--P4--|---P1---|---P3---|----P2----|

Waiting Time

Process	Waiting Time
P4	0
P1	3

Process	Waiting Time
---------	--------------

P3	9
----	---

P2	16
----	----

Average Waiting Time

$$= (0 + 3 + 9 + 16) / 4 = \mathbf{7\ ms}$$

Turnaround Time

Process	Turnaround Time
---------	-----------------

P4	3
----	---

P1	9
----	---

P3	16
----	----

P2	24
----	----

Average Turnaround Time

$$= (3 + 9 + 16 + 24) / 4 = \mathbf{13\ ms}$$

Advantages

- Minimum average waiting time.
- Better CPU utilization.

Disadvantages

- Burst time prediction is difficult.
- Long processes may starve.

3. Compare FCFS, SJF, Priority and Round Robin Scheduling Algorithms

Feature	FCFS	SJF	Priority	Round Robin
Scheduling Basis	Arrival Time	Shortest Burst Time	Highest Priority	Time Quantum
Type	Non-preemptive	Preemptive/Non-preemptive	Preemptive/Non-preemptive	Preemptive
Average Waiting Time	High	Lowest	Moderate	Moderate
Starvation	No	Yes	Yes	No
Response Time	Poor	Good	Good	Excellent
Context Switches	Low	Low	Medium	High
Suitable For	Batch Systems	Short Jobs	Real-Time Systems	Time-Sharing Systems

4. Solve a Scheduling Problem Using FCFS and Calculate Average Waiting Time and Turnaround Time

Given

Process	Arrival Time	Burst Time
P1	0	4
P2	1	5
P3	2	2
P4	3	1

Gantt Chart

0 4 9 11 12
|--P1--|---P2----|---P3--|---P4--|

Waiting Time

Process	Waiting Time
P1	0
P2	$4 - 1 = 3$
P3	$9 - 2 = 7$
P4	$11 - 3 = 8$

Average Waiting Time

$$= (0 + 3 + 7 + 8) / 4$$

$$= \mathbf{4.5 \text{ ms}}$$

Turnaround Time

Process	Turnaround Time
P1	4
P2	8
P3	9
P4	9

Average Turnaround Time

$$= (4 + 8 + 9 + 9) / 4$$

$$= 7.5 \text{ ms}$$

5. Describe Inter-Process Communication (IPC) Methods with Neat Diagrams

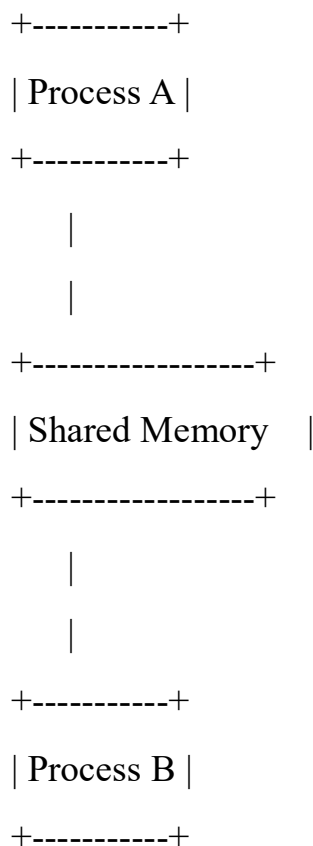
Definition

Inter-Process Communication (IPC) is a mechanism that allows processes to exchange information and synchronize their execution.

IPC Methods

1. Shared Memory

Two or more processes share a common memory region.



Advantages

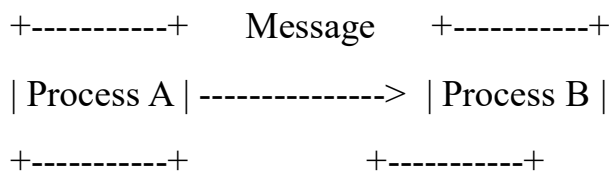
- Fast communication.
- Efficient for large data.

Disadvantages

- Requires synchronization mechanisms such as semaphores or mutexes.
-

2. Message Passing

Processes communicate by sending and receiving messages.



Advantages

- No shared memory required.
- Easier to implement in distributed systems.

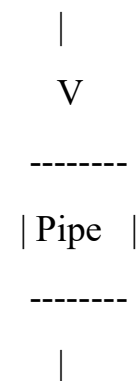
Disadvantages

- Slower than shared memory.
-

3. Pipes

A pipe provides a communication channel between related processes.

Process A



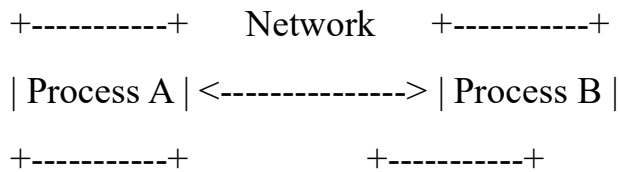
V

Process B

Used for: Parent-child process communication.

4. Sockets

Sockets enable communication between processes on the same or different computers over a network.



Applications

- Client-server systems
 - Web applications
-

5. Signals

Signals are software interrupts used to notify a process of an event.

Process A

|

Signal

|

V

Process B

Examples

- Interrupting a process
- Terminating a process
- Handling exceptions

Comparison of IPC Methods

IPC Method	Speed	Communication Type	Typical Use
Shared Memory	Very Fast	Shared Memory	Large data sharing
Message Passing	Moderate	Messages	Distributed systems
Pipes	Fast	Byte Stream	Parent-child communication
Sockets	Moderate	Network Communication	Client-server applications
Signals	Fast	Notifications	Event handling

PART-C

1. Five Processes Arrive at Different Times with Different Burst Times. Draw the Gantt Chart and Calculate Average Waiting Time and Turnaround Time Using FCFS, SJF, Priority Scheduling and Round Robin. Compare the Performance of All Algorithms.

Given

Process Arrival Time Burst Time Priority (*1 = Highest*)

P1	0	8	2
P2	1	4	1
P3	2	9	4
P4	3	5	3
P5	4	2	5

(a) FCFS Scheduling

Gantt Chart

0 8 12 21 26 28
|---P1---|--P2--|----P3-----|--P4--|P5|

Calculations

Process WT TAT

P1	0	8
P2	7	11
P3	10	19
P4	18	23
P5	22	24

Average Waiting Time = $(0 + 7 + 10 + 18 + 22) / 5 = 11.4$ ms

Average Turnaround Time = $(8 + 11 + 19 + 23 + 24) / 5 = 17$ ms

(b) Non-Preemptive SJF Scheduling

Execution Order

P1 → P5 → P2 → P4 → P3

Gantt Chart

0 8 10 14 19 28
|---P1---|P5|--P2--|--P4--|----P3-----|

Calculations

Process WT TAT

P1	0	8
----	---	---

Process WT TAT

P5	4	6
P2	9	13
P4	11	16
P3	17	26

Average Waiting Time = 8.2 ms

Average Turnaround Time = 13.8 ms

(c) Priority Scheduling (Non-Preemptive)

Lower number indicates higher priority.

Execution Order

P1 → P2 → P4 → P3 → P5

Gantt Chart

0 8 12 17 26 28
|---P1---|--P2--|--P4--|----P3-----|P5|

Calculations**Process WT TAT**

P1	0	8
P2	7	11
P4	9	14
P3	15	24
P5	22	24

Average Waiting Time = 10.6 ms

Average Turnaround Time = 16.2 ms

(d) Round Robin Scheduling

Time Quantum = 2 ms

Gantt Chart

0 2 4 6 8 10 12 14 16 18 20 22 24 26 28

|P1|P2|P3|P1|P4|P5|P2|P3|P1|P4|P3|P1|P4|P3|

Final Results

Process WT TAT

P1 14 22

P2 9 13

P3 17 26

P4 16 21

P5 6 8

Average Waiting Time = 12.4 ms

Average Turnaround Time = 18 ms

Comparison of Scheduling Algorithms

Algorithm	Avg Waiting Time	Avg Turnaround Time	Advantages	Disadvantages
FCFS	11.4 ms	17 ms	Simple	Convoy effect
SJF	8.2 ms	13.8 ms	Lowest waiting time	Starvation of long jobs
Priority	10.6 ms	16.2 ms	Important jobs execute first	Starvation of low- priority jobs

Algorithm	Avg Waiting Time	Avg Turnaround Time	Advantages	Disadvantages
Round Robin	12.4 ms	18 ms	Fair and responsive	Many context switches

Conclusion

- **SJF** gives the **lowest average waiting time and turnaround time**.
- **Round Robin** is most suitable for **time-sharing systems** because every process gets CPU time fairly.
 - **Priority Scheduling** is useful in **real-time systems**.
- **FCFS** is the simplest but generally provides poorer performance than SJF.

2. Explain Race Condition with a Banking Application Example. How Can Synchronization Eliminate Race Conditions?

Definition

A **Race Condition** occurs when two or more processes or threads access and modify the same shared data simultaneously, causing unpredictable or incorrect results.

Banking Application Example

Assume a bank account has a balance of **₹10,000**.

Two customers perform transactions simultaneously:

- **Customer A withdraws ₹4,000**
- **Customer B withdraws ₹7,000**

Without Synchronization

Both processes read the balance at the same time:

Initial Balance = ₹10,000

Thread A reads ₹10,000

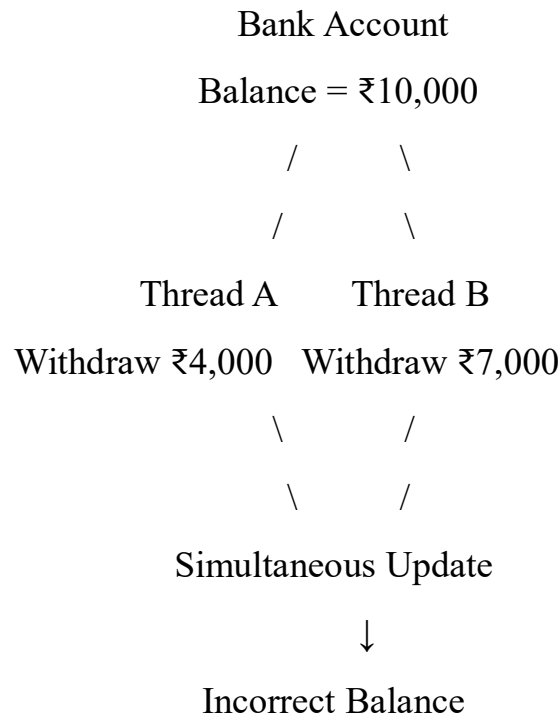
Thread B reads ₹10,000

Thread A writes ₹6,000

Thread B writes ₹3,000

The final balance becomes **₹3,000**, even though a total of **₹11,000** was withdrawn from an account that only had **₹10,000**. This is an incorrect result caused by a race condition.

Diagram



Critical Section

The **critical section** is the part of the program where shared data (the bank balance) is accessed or modified.

Only one process should execute the critical section at a time.

Synchronization

Synchronization coordinates processes or threads so that only one can access the shared resource at any given time.

Using a Mutex (Lock)

Thread A

|

Acquire Lock

|

Update Balance

|

Release Lock

|

Thread B waits

After Thread A releases the lock, Thread B enters the critical section and updates the balance safely.

Synchronization Techniques

- Mutex (Mutual Exclusion)
 - Semaphore
 - Monitor
 - Spinlock
- Critical Section Locks

Advantages of Synchronization

- Prevents race conditions.

- Ensures data consistency.
 - Maintains correct execution order.
 - Prevents data corruption.
 - Improves system reliability.
-

Conclusion

Race conditions occur when multiple processes or threads access shared data without proper coordination. In banking systems, this can lead to incorrect account balances and transaction errors. By using synchronization mechanisms such as **mutexes**, **semaphores**, or **monitors**, only one thread is allowed to execute the critical section at a time, ensuring data integrity and correct program execution.